Detecting bare-nested GTP-U tunnels that packet dissectors do not decode

Joseph Kwabena Fiagbor[a,*] , Oliver Kornyo[a]

[a] *Department of Computer Science, Kwame Nkrumah University of Science and Technology, Kumasi, Ghana*

ABSTRACT

GTP-U tunnels carry 5G subscriber traffic with no authentication or integrity protection. One abusive construction places a second GTP-U header directly after the outer one. We show from the source code of Scapy, Wireshark and Zeek that none of them decodes such a bare nested header: each identifies the payload of a G-PDU as IPv4 or IPv6 from the version nibble of its first byte (Wireshark also tries Ethernet) and has no path that decodes a nested GTP-U header, so a detector that asks whether the payload is itself GTP-U never sees it. The bare form requires an on-path position on N3 or a non-IP session type; the IP-wrapped form an ordinary session can send is decoded natively and is not the subject of the main claim. We describe a passive engine that re-dissects the payload behind a byte-level plausibility gate, after the extension-header chain that real N3 traffic carries, with four rules covering nested tunnels, tunnel endpoint identifier re-sourcing, control-plane smuggling and inner traffic aimed at core functions. On 13,203 synthetic packets from ten seeds held out of development, with realistic outer headers, it reaches recall 1.0 (95% interval 0.9994 to 1.0) and a false-positive rate of 0.0 (0 to 0.0005); without the extension-header handling it misses every bare-nested packet. On the independent University of Oulu 5G-CGAD captures it flags all 290,225 nested packets and none of 6,218,456 benign GTP-U packets from 30 subscribers, and it does not detect TEID brute force. Zeek flags only the IP-wrapped form and Suricata with 53,289 ET Open rules flags neither. Two fresh live sequences against an Open5GS and UERANSIM core with three subscribers produce finding streams identical to offline replay, detecting 998 of 1,000 attack packets with no false positive and no finding on real subscriber traffic. IPv6 inner coverage is partial. All code and data are released.

Keywords: 5G core network; GTP-U; user plane security; packet dissection; intrusion detection; reproducibility

1. Introduction

Fifth-generation standalone networks separate signalling from subscriber traffic. Signalling travels across a service-based architecture of network functions communicating over HTTP/2 with mutual authentication, while subscriber traffic travels across the user plane. Most of the security machinery the 3rd Generation Partnership Project (3GPP) has added since Release 15 sits on the signalling side.

The user plane inherited its transport from an earlier era. Subscriber packets move between the base station and the User Plane Function, and between User Plane Functions, inside GTP-U tunnels defined in TS 29.281 [1]. The header is eight bytes in its minimal form: a flags octet, a message type, a length field and a 32-bit tunnel endpoint identifier, with four optional bytes when the extension, sequence or network protocol data unit (N-PDU) flags are set. There is no sender authentication, no integrity check and no encryption. The protocol assumes that anything able to place a packet on the N3 or N9 interface is a trusted network element. That was reasonable when those interfaces sat inside a physically controlled backhaul. It is less reasonable now that cores are containerised, that User Plane Functions are deployed at the edge, and that private 5G networks are run by organisations that do not specialise in operating a mobile network.

A device holding a data session is a legitimate source of bytes inside a GTP-U tunnel, and it controls those bytes completely. If it emits a packet whose content happens to be another GTP-U header, the base station encapsulates it as it would encapsulate anything else, and what arrives at the User Plane Function is a tunnel inside a tunnel. Trend Micro reported in 2023 that this construction crashes the User Plane Function in a widely used open-source core, catalogued as CVE-2021-45462, and that a related technique delivers signalling to the Access and Mobility Management Function from user-plane traffic [2,3]. A separate disclosure showed that the absence of address cross-checking between planes lets an attacker reachable from N3 reflect packets at devices inside the network [4].

Given that these techniques are public, detection for them ought to be routine, and an obstacle sits in the packet dissector. Scapy [5], Wireshark and Zeek decide what a user-data G-PDU (the GTP-U message type that carries a subscriber packet) contains by testing the version nibble of its first payload byte: 4 means IPv4, 6 means IPv6, and anything else is either passed to a non-IP guess or left undecoded (Section 7.3 gives the source-code evidence). None of the three has a path that decodes a GTP-U header placed directly after the outer one, whose first byte is 0x3X. A detector asking whether the payload is itself GTP-U gets a truthful answer about the outer header and learns nothing about the inner one, and no error is reported. Section 2.3 separates this bare form from an IP-wrapped variant that the same dissectors decode without difficulty, and the two differ in reachability: the IP-wrapped form is available to an ordinary subscriber session, while the bare form requires an on-path position on N3 or an Unstructured or Ethernet session type (Section 4).

We treat that behaviour as a detection problem in its own right. The engine described here recovers the payload bytes after the outer header and its extension headers, re-dissects them when, and only when, their leading bytes pass a plausibility gate for a GTP header, and confirms that the recovered header forwards a routable inner packet before raising anything. Without the gate the detector would re-interpret every payload; without the guard it would flag benign unstructured session data that happens to collide with the GTP byte pattern.

The evaluation is built to limit the circularity of scoring rules against attacks written by the same authors. Development used one seed family and one live sequence; every reported held-out result uses seeds, captures or live sequences that played no part in changing the code. Those include the independent 5G-CGAD dataset from the University of Oulu [25], open-source comparators, and live runs with several subscribers.

The main contribution is the first item below; the others support it.

- Main: it shows from source code and measurement that default dissection in Scapy, Wireshark and Zeek never decodes a bare-nested GTP-U header, separates that case from the IP-wrapped form, and specifies a targeted re-parse, with an explicit plausibility gate, a routable-inner guard and extension-header handling, that recovers it.
- Supporting: it evaluates a four-rule engine on development data, on held-out seeds with realistic N3 framing, on the independent 5G-CGAD captures and in live runs with three subscribers, and reports intervals that reflect the number of packets scored.
- Supporting: it compares the engine with a naive baseline that differs only in the re-parse, and with Zeek and Suricata on identical captures.

- Supporting: it releases a containerised testbed, a seeded generator with per-packet labels, an evasion suite and its documented blind spots.

Section 2 covers GTP-U and why it is exposed. Section 3 reviews recent work, including operational GTP inspection, and states three gaps. Section 4 gives the threat model and scope. Section 5 describes the testbed, corpus and engine. Section 6 reports the offline evaluation and Section 7 the independent-data and live evaluation. Section 8 discusses limitations and Section 9 concludes.

2. Background

2.1. GTP-U in the 5G user plane

Fig. 1 shows where GTP-U operates. A device attaches over the radio interface and is assigned a session. Its IP packets are encapsulated by the base station and sent over N3 to a User Plane Function, which decapsulates and forwards them towards the data network over N6. Where a session is anchored at a second User Plane Function the tunnel runs between them over N9. Session state is established by the control plane over N2 and N4, but the control plane does not inspect tunnelled bytes; doing so would defeat the point of separating the planes.

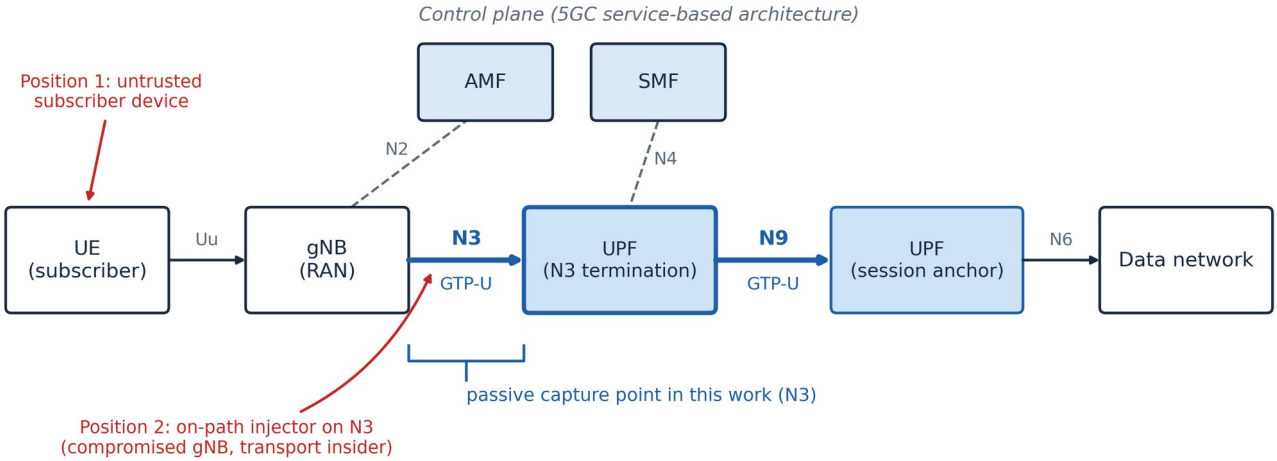


Fig. 1. The 5G standalone user plane. GTP-U tunnels carry subscriber traffic across N3 and N9. Two adversary positions are marked (Section 4). An untrusted device sends packets that an honest base station encapsulates, so they reach the User Plane Function inside a well-formed tunnel from an authenticated peer. An on-path injector on N3 composes tunnel headers itself. The passive capture point is N3.

Two properties matter here. The tunnel endpoint identifier (TEID) is the only field distinguishing one subscriber's tunnel from another's, and it is a plain 32-bit value carried in clear. And the encapsulating node performs no inspection of what it encapsulates: the base station does not ask whether the bytes a device handed it resemble a subscriber packet.

2.2. Why the protocol is structurally exposed

TS 33.501 recommends IPsec on N3, and the GSM Association (GSMA) has published user-plane specific guidance [6,7]. In practice the recommendation is often skipped, because IPsec on the backhaul costs latency and throughput at exactly the point where 5G is sold on latency and throughput. Recent surveys of 5G security, spanning the physical layer through the core, report the same pattern: protections exist in the specifications and are unevenly deployed [8,9,10].

Even where IPsec is present it protects the segment, not the content. It authenticates the base station to the User Plane Function and says nothing about whether the bytes the base station faithfully forwarded were well formed. Abuse originating at a subscriber device and legitimately encapsulated by an honest base station passes through an IPsec tunnel unchanged. That is the first of the two adversary positions considered here (Section 4), and it is why transport protection is not a substitute for inspection. The second, an on-path injector without keys, presupposes an N3 segment without IPsec, which as noted above is common in practice.

IPsec also decides where a sensor can sit. A passive tap reads GTP-U headers and payloads, so it needs N3 in clear: either on a segment where IPsec is not deployed, or behind the point where IPsec terminates, such as the User Plane Function host or the inner side of a security gateway. On an encrypted segment a tap sees only ESP and can parse nothing. Our testbed places the sensor in the User Plane Function's own network namespace (Section 5.1), which corresponds to the second option.

2.3. Abuse classes

Fig. 2 contrasts a conformant GTP-U packet with the constructions that motivate this work. Conformant traffic yields a subscriber IP packet after one decapsulation. In the smuggling case the payload is a signalling message addressed at a control-plane node, crossing a boundary it was never meant to cross.

Nesting has two forms, and the distinction matters for detection. In the bare form, Fig. 2(b), a GTP header follows the outer one directly. In the IP-wrapped form, Fig. 2(c), the inner tunnel is a complete datagram with its own IP and UDP headers. The two are equivalent as attacks and are not equivalent to a dissector: the wrapped form carries a complete inner IP datagram, whose first byte begins a valid IPv4 header and therefore decodes natively; the dissector then reaches the inner UDP GTP-U port and recovers the second tunnel. The bare form presents a GTP-U header as that first byte, which the first-byte test does not recognise, so the nested header is never decoded as GTP-U. Section 6.2 measures the consequence. Only the bare form appears in our corpus, and the reachability of each form is discussed in Section 8.

A third construction leaves the encapsulation depth alone and changes where the inner packet points. Instead of the data network, the inner destination belongs to core infrastructure. Because the User Plane Function decapsulates before routing, an inner packet aimed at a core address reaches somewhere a subscriber has no route to from outside the tunnel. This is the mechanism behind the reflection class in [4].

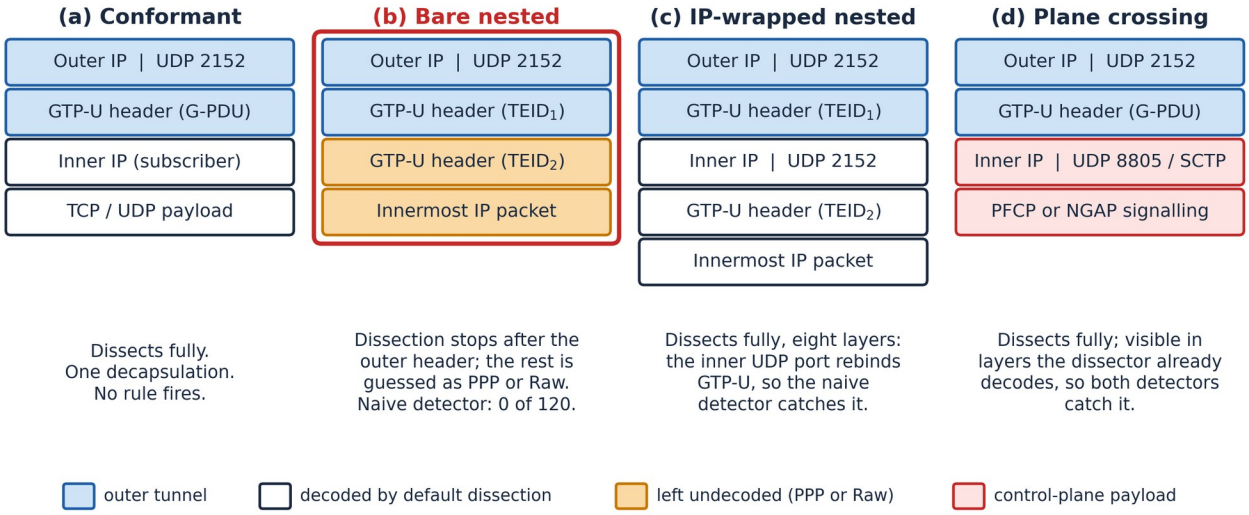


Fig. 2. Conformant GTP-U compared with three abusive constructions. Nesting takes two forms. In the bare form (b) the second tunnel header follows the outer one directly, and default dissection does not recognise it as GTP-U: it is guessed as PPP under a G-PDU outer header, or left as Raw under an outer header with message type 0x00. This is the blind spot this paper measures. In the IP-wrapped form (c) the payload begins with a valid inner IP header, so dissection proceeds to the inner GTP-U port and recovers all eight layers, and a naive detector sees the attack. Plane crossing (d) is likewise visible by default. Only construction (b) appears in the corpus.

3. Related work and research gaps

3.1. Vulnerability disclosure and protocol analysis

The most directly relevant work is a sequence of industrial disclosures. Salim reported that a zero-length GTP-U message of type 255 delivered inside a GTP-U tunnel denies service in the User Plane Function of an open-source core [2], and subsequently that anomalous signalling delivered through the user plane crashes the Access and Mobility Management Function of another [3]. A joint disclosure with CTOne and the Telecom Technology Center described a packet reflection vulnerability rated 8.3 on the Common Vulnerability Scoring System, arising from missing address cross-checks between planes [4]. These establish that the abuse classes are real and affect multiple implementations. They are advisories: they demonstrate exploitation and recommend mitigation at the level of firewall policy and vendor patching, without proposing or evaluating a detector. Systematic reviews of control- and user-plane weaknesses reach the same point from the survey side, mapping vulnerabilities to specific network functions and interfaces and setting out solution requirements, but likewise stopping short of a detector [11,12].

Fuzzing approaches the same surface from the other side. 5Greplay injects malformed 5G traffic to test core robustness [13], and coverage-guided fuzzers have since been applied to multi-component 5G stacks. These tools find defects; they do not tell an operator that an attack is under way.

3.2. Testbeds and dataset generation

A body of recent work builds experimental 5G environments from open-source components, usually pairing Open5GS with the UERANSIM radio access network emulator [14,15]. Comparative evaluations of these platforms exist [16,17,18], and modular laboratory frameworks now lower the effort of standing one up [19]. Our testbed follows this practice and claims no novelty in its construction.

Dataset generation has followed. Amponis and colleagues built an integrated testbed on the same two components and released generators for persistent session attacks against internal core interfaces, concentrating on the Packet Forwarding Control Protocol (PFCP) [20], having earlier characterised denial of service against it [21]. The 5G-NIDD dataset, produced on a comparable testbed, is now a common benchmark [22,23]. The emphasis throughout is volumetric and signalling attacks. Where the user plane appears it supplies flow features for classification rather than being examined as a protocol whose encapsulation semantics are themselves at issue [24].

Closest to this work is the 5G Core GTP-U Attack Dataset (5G-CGAD) from the University of Oulu [25]. It was generated on a Kubernetes testbed with Open5GS, 30 UERANSIM subscribers and three base stations, and provides packet captures and labelled flow features for benign traffic and five GTP-U attack scenarios: GTP encapsulation, malformed GTP, TEID brute force, intra-UPF denial of service and flooding. Labels are given per scenario and per flow. Its nested attack is the IP-wrapped form sent by a subscriber; it contains no bare-nested packet. Section 7.1 evaluates the engine on it.

3.3. Learning-based detection

Detection work in this area is dominated by machine learning. Feature selection has been applied to GTP-U flow records to cut the latency of classifying distributed denial of service traffic [24], and supervised classifiers have been benchmarked on 5G-NIDD [23]. These report strong headline metrics. Their shared assumption is that abuse shows up as a statistical deviation in aggregate flow behaviour.

That assumption is less natural for the classes considered here. A single nested GTP-U packet is not a volumetric event, and the evidence that distinguishes it lies in header bytes that a flow-feature pipeline summarises away. We did not run a learned or flow-based detector, so this is an argument rather than a measurement; whether flow features separate nested packets in practice is left open (Section 8).

Operational GTP inspection is the other relevant strand. Commercial GTP firewalls inspect the user plane: Juniper documents a GTP-in-GTP check that drops a GTP-U packet whose payload is itself a GTP packet [26], Fortinet states that its carrier firewall cannot open a GTP tunnel nested in another and can only block or forward it [27], and Palo Alto Networks offers GTP-U validity checks and content inspection [28]. These are closed implementations, and their handling of the bare form cannot be verified independently. Among open tools, Zeek decapsulates GTPv1 and reports a GTP-in-GTP analyzer violation for the IP-wrapped form [29], Suricata has no GTP-U decoder [30], and Snort 3's

gtp_inspect covers the GTP control channel [31]. Section 6.4 runs Zeek and Suricata on the same captures as the engine.

Table 1 places the present work against these strands. The columns record what each item is, whether it handles the bare nested form, whether its behaviour can be checked by a third party, how its data are labelled, and whether it was assessed on data its authors did not generate.

3.4. Research gaps

Read together, these strands leave three gaps. Each is stated narrowly enough to be closed or refuted by evidence.

Gap 1. The handling of bare nested encapsulation by open dissectors and intrusion detection tools has not been examined. Commercial firewalls advertise GTP-in-GTP checks, but their behaviour is not open to inspection, and the open tools a researcher or operator can audit have not been shown either to decode a bare second tunnel header or to miss it.

Gap 2. Public GTP-U attack data now exist [25], but they label traffic per scenario and per flow, and they do not contain the bare nested form. No public artefact ties per-packet labels to the construction of each packet for that form.

Gap 3. Detectors in this domain are usually scored against attacks built to match their own rules, with no deterministic baseline and no adversarial probing. A perfect score obtained that way carries little information. Our own development corpus has the same weakness, which is why the evaluation adds held-out seeds, independent captures and live runs.

3.5. Research objectives

The gaps translate into five objectives, mapped in Table 2.

Table 1

Positioning against recent work and operational tools. A dash marks a dimension that does not apply or was not reported.

Work	What it is	Bare nested form	Third-party verifiable	Labels	Assessed on others' data
Salim [2,3,4]	Vulnerability disclosures	Shown as an attack	—	—	—
5Greplay [13]	Fuzzer for 5G cores	—	Open source	—	—
Amponis et al. [20]	Dataset: PFCP and session attacks	Not covered	Open data	Flow	—
5G-NIDD [22]	Dataset: volumetric and signalling	Not covered	Open data	Flow	—
5G-CGAD [25]	Dataset: GTP-U attacks	Not covered (IP-wrapped only)	Open data	Scenario, flow	—
Kim et al. [24]	Learned detection on GTP-U flows	Not examined	—	Flow	Not reported
Juniper SRX [26]	Commercial GTP firewall	GTP-in-GTP check documented	No	—	Vendor-stated
Fortinet [27]	Commercial GTP firewall	Cannot open; blocks or forwards	No	—	Vendor-stated
Zeek 8.2.2 [29]	Open network monitor	Not decoded (Section 6.4)	Yes	—	—
Suricata 8.0.7 [30]	Open IDS	No GTP-U decoder (Section 6.4)	Yes	—	—
This work	Engine, testbed, generator	Detected (Sections 6, 7)	Yes	Per packet	Yes: 5G-CGAD (Section 7.1)

Table 2

Research objectives and the gaps they address.

Objective	Addresses
O1. Characterise default dissection on nested GTP-U and quantify the resulting loss of recall.	Gap 1
O2. Specify a targeted re-parse with an explicit plausibility gate and a routable-inner guard.	Gap 1
O3. Produce a reproducible testbed and seeded generator yielding per-packet labelled captures.	Gap 2
O4. Evaluate a deterministic four-rule engine against a naive baseline and open tools, on held-out and independent data.	Gap 3
O5. Probe the engine with an evasion suite and publish the blind spots it exposes.	Gap 3

4. Threat model and scope

We consider two adversary positions. The first, the position the published disclosures describe, controls a subscriber device that has completed registration and holds a valid session. It is an ordinary customer, not a privileged insider. It can compose arbitrary IP packets on its own interface, which is within reach of any rooted handset or device presenting as a modem. It cannot modify base station firmware, obtain keys, place packets directly on N3 or N9, or read other subscribers' tunnels.

The second position is on-path between the base station and the User Plane Function, held by a compromised base station or an insider with access to the transport segment. It holds no subscriber session and composes tunnel headers directly. We assume only that it can place packets on N3; we do not assume it can obtain keys, read other subscribers' tunnels or modify the core.

The two positions reach different constructions, and the distinction bears on the results in Section 6. Control-plane smuggling, inner-to-core traffic and the IP-wrapped nested form are all reachable from the subscriber position, since each is expressible as an IP packet inside a payload the device controls. The bare nested form is not. Nor is TEID re-sourcing, since the subscriber never writes the outer header; it too requires the on-path position, which is the position the testbed emulates (Section 5.1). On an IPv4 or IPv6 session the base station encapsulates the subscriber's packet whole, so the first byte after the outer header is necessarily 0x4X or 0x6X. Emitting a bare nested header therefore requires either the on-path position or an Unstructured or Ethernet session type, where the payload is not constrained to be IP. We state this here rather than only in Section 8, because the ablation in Section 6.2 and the R1 result rest on the bare form.

The subscriber adversary's leverage is that it chooses the payload of a tunnel granted to it legitimately. The base

station encapsulates whatever packet it sends, so that payload arrives at the User Plane Function inside a well-formed outer tunnel from an authenticated peer. The on-path adversary writes the outer header itself.

Three objectives follow: reaching an implementation defect in the core's parsing of tunnelled data; delivering a message to a network function it has no route to, crossing from user plane to control plane; or reaching another device or a management address by choosing an inner destination the User Plane Function will route after decapsulation.

We assume a passive capture point on the segment between the base station and the User Plane Function, placed where N3 is visible in clear (Section 2.2). We do not assume the ability to block. Detection that cannot be trusted to block is still useful; detection that blocks and is wrong takes subscribers off the network.

Scope. The engine inspects GTPv1-U over IPv4 or IPv6 transport. R1 covers nested tunnels that forward IPv4 or IPv6, and R3's port checks cover both inner IP versions, but R3's SCTP check and R4 inspect IPv4 inner packets only, so SCTP smuggling and core targeting inside an IPv6 inner packet are out of scope. The engine does not inspect GTP-C or PFCP traffic on their own interfaces.

5. System and methodology

5.1. Testbed

The environment, shown in Fig. 3, runs as containers on a single host under Docker Compose. Open5GS provides the core network functions including the User Plane Function terminating N3. UERANSIM emulates the base station and a configurable number of subscriber devices; each completes the normal registration procedure and requests an IPv4 PDU session, so the sessions' own tunnels are real rather than synthesised. The development host ran one subscriber; the live runs of Section 7.2 ran three, on a separate cloud host. A separate attacker container composes crafted packets with Scapy. It is not a subscriber device and holds no session: it runs in the core container's network namespace and writes each packet with a raw socket onto the core's N3 interface, addressed to the User Plane Function. The outer source address is the base station's for every class except TEID re-sourcing, which uses an address no base station holds. UERANSIM does not encapsulate these packets, so the attacker stands in for the on-path position of Section 4; for the classes a subscriber can reach, it emits the N3 form those packets take once an honest base station has encapsulated them. The Unstructured and Ethernet session types were not exercised. The detector observes the crafted frames on N3 as a passive tap would; whether the User Plane Function acts on them was not measured. Component versions are listed in Table 3.

Table 3

Component versions. The development host produced the offline results of Section 6; the cloud host produced the 5G-CGAD and live results of Section 7.

Component	Version
Development host	Ubuntu 22.04.5, kernel 6.8.0-138-generic, Intel Core i7-6820HQ (virtual machine); Python 3.10.12, Scapy 2.4.4
Cloud host	Ubuntu 26.04.1, kernel 7.0.0-1014-aws, 2 vCPU Intel Xeon Platinum 8488C; Python 3.14.4, Scapy 2.7.0
Docker Engine and Compose	29.1.3 and v5.3.1 (development); 29.9.0 and v5.6.0 (cloud)
Open5GS	2.8.0~jammy5 (ppa:open5gs/latest) on both hosts
UERANSIM	v3.2.6 (pinned via .env)
MongoDB	7 (mongo:7 image; 7.0.45 on the cloud host)
Detector container	Python 3.12, Scapy 2.7.0 (detector/requirements.txt)
Comparators	Zeek 8.2.2; Suricata 8.0.7 with ET Open (53,289 rules); Wireshark tshark 4.6.6
Test public land mobile network	Mobile country code 999 / mobile network code 70 / tracking area code 1

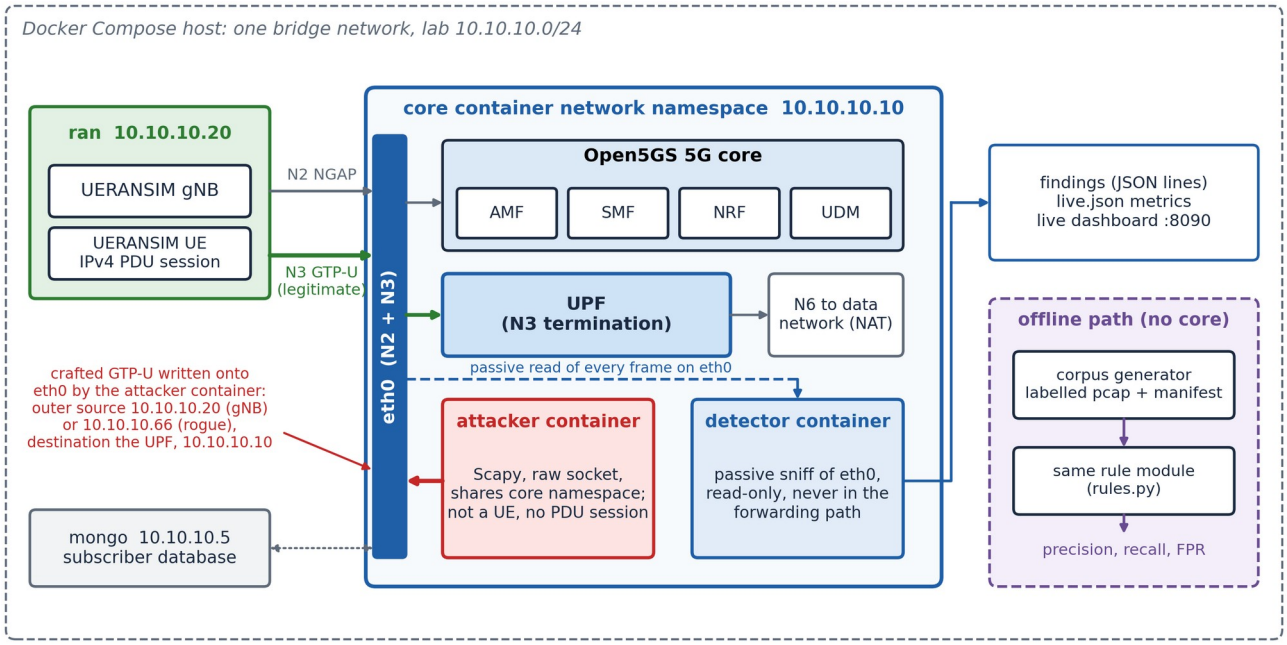


Fig. 3. Testbed topology. Open5GS provides the core and UERANSIM establishes a genuine IPv4 session over N2 and N3. The attacker container is not a subscriber device: it shares the core's network namespace and writes crafted GTP-U onto the N3 interface with the base station's or a rogue outer source address, standing in for the on-path position of Section 4. The detector shares the same namespace and reads every frame on that interface passively; it is never in the forwarding path. The offline path replays a labelled capture through the same rule module.

Two layout decisions are deliberate. The detector shares the core container's network namespace, so it observes the User Plane Function's actual interface rather than a mirrored copy, and it is never in the forwarding path, so a false positive costs an alert rather than a dropped session. And the live path carries genuine subscriber traffic beside the labelled packets, whose tunnel endpoint identifiers, addressing and header flags are those the core negotiated. The scored benign and malicious packets are synthesised by the seeded generator.

5.2. Corpus and labelling

The generator writes a capture together with a per-packet manifest recording each packet's class label. Labels are assigned at synthesis time, when construction is known exactly, rather than inferred afterwards from a time window. Generation is seeded and the seed recorded, so a given seed reproduces a corpus byte for byte; a regression test enforces this and also checks that a different seed produces a different corpus.

The primary corpus, at seed 1337, holds 1,320 packets: 600 malicious spread evenly across five attack classes at 120 each, and 720 benign. The benign portion is 600 packets drawn across twelve categories, weighted towards what dominates real user planes, plus 120 legitimate victim flows that establish tunnel endpoint ownership before a spoof re-uses it. Table 4 gives the composition. The five malicious classes are those the released generator emits: gtp_in_gtp, teid_spoof, pfcp_smuggle, ngap_smuggle and inner_to_core. The class named ngap_smuggle builds an inner packet carrying IP protocol 132, the Stream Control Transmission Protocol (SCTP) over which NG Application Protocol (NGAP) signalling runs. We retain the generator's name for it so that the paper and the artefact agree. Four benign categories are adversarial by design: unstructured protocol data unit (PDU) session bytes, the natural false-positive source for any byte-pattern heuristic; fragmented IP and IP options, which shift offsets; and handover, which legitimately moves a tunnel endpoint identifier between base stations and is the natural false-positive source for R2.

Table 4

Composition of the primary labelled corpus at seed 1337.

Portion	Packets	Composition
Malicious	600	gtp_in_gtp, teid_spoof, pfcp_smuggle, ngap_smuggle, inner_to_core (120 each)
Benign, categories	600	web_tls, quic, dns, web_http, rtp_voip, ipv6, icmp, fragmented, ip_options, unstructured, ntp, handover
Benign, victim flows	120	legitimate traffic establishing TEID ownership prior to a spoof
Total	1,320	720 benign / 600 malicious

5.3. The targeted re-parse

This is the part of the system that carries the argument, and its shape is shown in Fig. 4. A frame is admitted if it carries a GTP-U layer. The engine then locates the payload after the outer header and its extension headers. A real N3 G-PDU sets the extension flag and carries a PDU Session Container, and each extension header states its own length and the type of the next, so the payload is found by walking that chain rather than by skipping a fixed number of bytes; a chain that runs past the end of the packet yields no payload. If the dissector has already decoded the payload as IPv4, IPv6 or a GTP header, the engine uses that decoding as it stands.

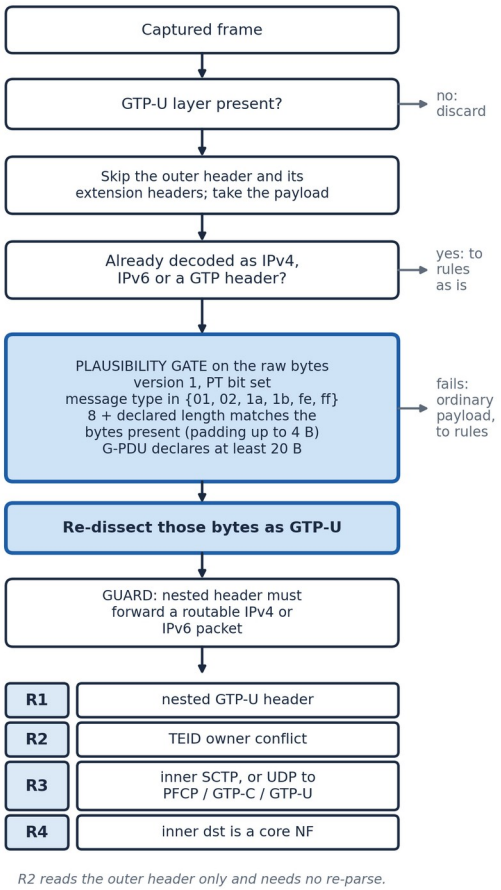


Fig. 4. Detection pipeline. The payload is located after the outer header and its extension headers. The re-parse is targeted rather than universal: it engages only on a payload the dissector has not decoded as IP or GTP, only behind a byte-level plausibility gate, and raises R1 only if the recovered header forwards a routable inner packet.

Otherwise the engine works on the payload's raw bytes rather than on the class the dissector assigned to them. The class is not a reliable signal: Scapy guesses a nested header as PPP inside a G-PDU and leaves it as Raw under an outer header with message type 0x00, so a re-parse keyed on one class would miss the other. Before re-dissecting, the engine applies the byte-level plausibility gate whose checks and parameters are listed in Table 5: GTP version 1, the protocol type bit, one of six message types, a length field consistent with the bytes present, and a minimum length for a G-PDU.

Payloads passing the gate are re-dissected as a GTP-U header. A second guard then applies: the recovered header must forward a routable inner packet, established either by the re-dissection yielding an IPv4 or IPv6 layer, or by the first nibble of the remaining bytes being 4 or 6.

Table 5

Parameters of the re-parse. All checks are on the bytes after the outer header and its extension headers; each must pass before R1 can fire.

Check	Condition	Purpose
Version and type	Top three bits of byte 0 equal 1; protocol type bit (0x10) set	GTPv1, not GTP prime
Message type	Byte 1 is 0x01 (echo request), 0x02 (echo response), 0x1A (error indication), 0x1B (supported extension headers), 0xFE (end marker) or 0xFF (G-PDU)	Only types valid on GTP-U
Length	8 + declared length (bytes 2-3) is at most the bytes present and at least that minus 4	Rejects buffers that cannot back their own header; tolerates up to 4 bytes of padding
G-PDU size	Declared length of a 0xFF or 0xFE message is at least 20 bytes	A G-PDU must carry at least an IPv4 header
Routable inner (guard)	Re-dissection yields IPv4 or IPv6, or the next byte has version nibble 4 or 6	The nested tunnel forwards a packet

Both guards exist because 5G Unstructured PDU sessions carry arbitrary non-IP bytes, some of which begin with the GTP byte pattern. A gate on version, type bit and message type alone flags such payloads; requiring the length field to be self-consistent and the nested header to carry something routable removes that class, with no false positive across 200 generated unstructured payloads, 93 of which deliberately start with a GTP-looking prefix, and no loss of recall.

The re-parse is single-level and targeted, not an unbounded recursive descent. It recovers one nested header that forwards a routable inner packet, which is the construction the corpus contains. The zero-length nested G-PDU of the disclosure in [2] is a different case: it carries no inner packet, so the length gate rejects it by design and, in the bare form, no rule fires; in the IP-wrapped form R3 still flags it as GTP-U inside the tunnel. Because the engine does not rely on the dissector's own payload binding, the same rule module gives identical findings under two Scapy releases (Section 7.3).

5.4. Detection rules

Four rules are specified in Table 6. Each corresponds to a condition a conformant user-plane packet cannot satisfy, which is why they can be decided without a trained model. Rules are evaluated independently and a packet may trigger several. The pfcp_smuggle and ngap_smuggle classes are each addressed at the User Plane Function, so they raise R3 and R4 together; that overlap, and not any interaction involving R1, is why the finding count exceeds the malicious packet count in Section 6. Any exception raised while evaluating a rule is contained, so a malformed packet cannot take the detector down.

Table 6

Detection rules and the condition each decides. Preconditions are given in the text.

ID	Name	Condition
R1	Nested tunnel	A GTP header recovered from the payload forwards a routable inner packet (Table 5).
R2	TEID re-sourcing	A tunnel endpoint identifier, scoped to the address that receives it, arrives from a source other than its owner.
R3	Control-plane smuggling	The inner packet carries IP protocol 132 (SCTP), or UDP to port 8805 (PFCP), 2123 (GTP-C) or 2152 (GTP-U).
R4	Inner-to-core	The inner destination is in the configured set of core network function addresses.

Each rule rests on a stated precondition. R1 relies on the fact that a subscriber payload is user data, so a second tunnel header has no legitimate origin at a device. R2 relies on the receiving side allocating endpoints per session: the first source seen for an identifier owns it, a packet from any other source raises R2 and leaves ownership unchanged, and, when an allowlist of base station addresses is configured, an allowlisted base station always takes ownership without a finding, which covers a handover and a base station reclaiming a tunnel a rogue was seen on first. R2 reads the outer header only, so it needs no re-parse. R3 relies on signalling belonging on N2 and N4, and is keyed on ports and protocol. R4 knows only the addresses it is given. Section 6.5 measures what R2 costs without the allowlist.

5.5. Evaluation protocol

Evaluation is per packet against the manifest. A true positive is a malicious packet on which any rule fires, a false positive a benign packet on which any rule fires, a false negative a malicious packet on which none does. Precision, recall, F1 and false-positive rate are computed overall, with recall per class. Every rate is reported with a Wilson 95% interval [32] over the number of packets scored, because the generator and the rules are deterministic and repeating a seed adds no independent evidence.

Development and evaluation data are kept apart. Development used seeds 1337, 1, 2, 3 and 4 and a single live sequence (seed 1337); two changes were made during development, to R2's ownership rule and to the extension-header handling, and both are in the version evaluated here. The held-out results use seeds 101 to 110, the independent 5G-CGAD captures, and two live sequences (seeds 2026 and 4242), none of which was used while the code was being changed. The extension-header handling was added after inspecting the header layout of the 5G-CGAD captures. Those captures contain only the IP-wrapped form, which the earlier version of the engine also flags in full on the capture we checked, so the change does not raise their scores; the held-out seeds were generated after it. Further checks are a unit suite of 25 tests, a naive-baseline ablation, a comparison with Zeek and Suricata, an evasion suite, and per-packet timing of dissection and rule evaluation together.

5.6. Operating modes and reproducibility

The system runs in two modes that share one rule module and one metrics collector. The offline mode needs no containers, no core and no elevated privileges: the generator writes a capture with an aligned label file, the detector replays it, and the harness scores the engine and the naive baseline. Separate scripts score held-out seeds, external captures and timing. It installs with two Python packages. The live mode brings up the full container stack and produces genuinely transmitted traffic.

Because there is one detection engine rather than two, the offline and live paths differ only in how packets reach the detector. The development results come from make eval, the unit tests from make test, and the per-packet scoring of a live run from make live-score; the held-out, external and timing results come from eval/heldout.py, eval/external_eval.py and eval/latency.py.

6. Offline evaluation

6.1. Development and held-out results

Table 7 gives the results. On the development corpus at seed 1337 the engine classified all 1,320 packets correctly, and so it did at the four other development seeds: 3,000 malicious and 3,600 benign packets with no error, which bounds recall below at 0.9987 and the false-positive rate above at 0.0011. Findings at seed 1337 totalled 840 across 600 malicious packets, 120 for R1, 120 for R2, 240 for R3 and 360 for R4; the excess reflects packets that satisfy more than one rule (Section 5.4). These seeds were used during development and are reported for continuity, not as evidence that the engine generalises. All 25 unit tests pass.

The held-out seeds 101 to 110 were generated once the code was frozen and give 13,203 packets, 6,000 malicious and 7,203 benign. With the plain outer header of the development corpus, the engine makes no error: recall 1.0 (95% interval 0.9994 to 1.0) and false-positive rate 0.0 (0 to 0.0005). A second held-out run adds a PDU Session Container extension header to the outer header of every packet, as a real N3 G-PDU carries, and changes nothing else. The result is the same. An engine without the extension-header handling, which takes the payload as the dissector returns it, scores recall 0.8 on that run: it misses all 1,200 bare-nested packets, because the dissector stops at the extension header and the engine never reaches the bytes behind it. The extension-header handling of Section 5.3 corrects this, and the plain-outer development corpus could not have shown it.

These scores need interpretation. The rules decide conditions that are violations by definition, and the generator builds those violations as specified. For R3 and R4 the relationship is close to tautological: the generator writes destination port 8805, IP protocol 132 or a core address, and the rules read exactly those fields, so recall of 1.0 on pfcp_smuggle, ngap_smuggle and inner_to_core shows that the corpus was built as specified rather than that detection is hard. R1 and R2 carry more information, R1 because the evidence has to be recovered before it can be read and R2 because it depends on state across packets. The comparison in Section 6.4 and the independent captures in Section 7.1 are the stronger tests.

Table 7

Development and held-out results, engine and naive baseline. Intervals are Wilson 95% over the packets scored. Precision was 1.0 in every row. At-risk benign packets are the unstructured and handover categories.

Corpus	Malicious / benign	Engine recall	Engine FPR	Baseline recall
Development, seed 1337	600 / 720	1.0 (0.9936-1.0)	0.0 (0-0.0053)	0.8
Development, seeds 1337, 1-4	3,000 / 3,600	1.0 (0.9987-1.0)	0.0 (0-0.0011)	0.8
Held-out 101-110, plain outer	6,000 / 7,203	1.0 (0.9994-1.0)	0.0 (0-0.0005)	0.8
Held-out 101-110, session container	6,000 / 7,203	1.0 (0.9994-1.0)	0.0 (0-0.0005)	0.8
Same, without extension-header handling	6,000 / 7,203	0.8 (0.790-0.810)	0.0 (0-0.0005)	0.8
At-risk benign, seed 1337	- / 23	-	0.0 (0-0.143)	-
At-risk benign, held-out	- / 334	-	0.0 (0-0.0114)	-

6.2. Naive-baseline ablation

The baseline implements the detector a competent engineer would write first. It keeps the four rule ideas, the state model and the corpus identical, and differs in exactly one respect: it uses only what the dissector produced, with no re-interpretation of a payload the dissector left undecoded or guessed as a non-IP class (Raw, PPP or any other). The delta therefore isolates a single variable.

The delta applies to a narrower case than the phrase “nested GTP-U” suggests. We ran both nested forms of Fig. 2 through both detectors, and Table 8 gives the result. The bare form, carried in a realistic G-PDU outer header as in the corpus, dissects to five layers, IP, UDP, GTP-U, PPP and Raw, because Scapy guesses the nested header as PPP; the baseline sees nothing. Under an outer header with message type 0x00 the same payload comes back as Raw and the layer count is four, which is why Section 5.3 keys the re-parse on the bytes and not on the class. The IP-wrapped form dissects to eight layers and the baseline catches it on three separate rules, because the inner UDP port rebinds GTP-U and hands the decoder everything it needs. The blind spot is a property of the bare form alone.

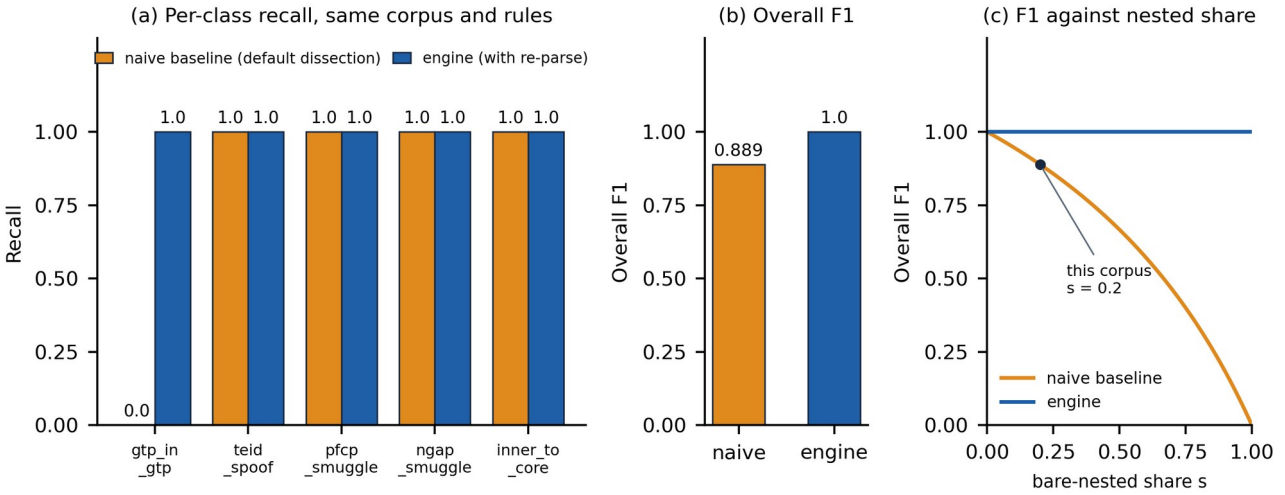


Fig. 5. Naive-baseline ablation. Corpus, rule logic and state model are identical; only the re-parse differs. (a) Per-class recall: the bare-nested class falls to zero and the other four are untouched. (b) Overall F1 falls from 1.0 to 0.889. (c) Baseline F1 as a function of the bare-nested share s of malicious packets, 2(1 - s)/(2 - s); the marker is the corpus share.

Table 8

The two nested constructions under default dissection, measured directly with a G-PDU (0xFF) outer header, the form the corpus uses. Only the bare form is opaque to the baseline. With an outer header of message type 0x00 the bare-nested payload decodes as IP / UDP / GTP-U / Raw (4 layers) and the outcome is the same.

Construction	How default dissection decodes it	Naive baseline	With re-parse
(b) bare nested, G-PDU outer [in corpus]	IP / UDP / GTP-U / PPP / Raw (5 layers)	no rule fires	R1
(c) IP-wrapped nested	IP / UDP / GTP-U / IP / UDP / GTP-U / IP / ICMP (8 layers)	R1, R3, R4	R1, R3, R4

On the corpus the result is in Fig. 5. Recall on the bare-nested class falls from 1.0 to 0.0: all 120 packets are missed, and the baseline reports no error while missing them, because it examined the frame, found a GTP-U layer and nothing anomalous in it. Recall on the other four classes stays at 1.0, since those classes are visible in layers the dissector already decodes. Overall F1 falls to 0.889 and recall to 0.8, with precision unaffected at 1.0. The F1 of 0.889 is a property of the corpus mix rather than of the baseline. With a share s of bare-nested packets among the malicious ones and every other packet scored correctly, the baseline's F1 is 2(1 - s)/(2 - s), plotted in Fig. 5(c); at the corpus share of 0.2 it is 0.889, and on traffic where the bare form is rare the overall F1 would barely move while every such packet was still missed. Per-class recall is the measure that matters.

The zero is deterministic rather than empirical. We first inferred from the layer stacks in Table 8 that the dissector chooses a G-PDU's inner protocol from the first payload byte, and then confirmed it in the source code of Scapy, Wireshark and Zeek (Section 7.3): none has a path that returns a GTP-U layer for a G-PDU payload, so once the bare form is in the corpus the baseline cannot return a recall other than 0.0. The measurement confirms a mechanism that can be read from the code; it does not discover one. Whether closed commercial implementations behave the same way cannot be checked from outside (Section 8).

6.3. Evasion suite

The evasion suite holds eleven crafted cases, each with a documented expectation of whether the engine should flag it, and the benchmark fails if behaviour departs from the expectation in either direction. Not all eleven are evasions. Seven try to slip past a rule: three of them, an IPv6 inner packet, an outer header with optional fields and a nested header that dissects as generic GTP, are caught, and four succeed and are recorded as blind spots. Three are positive controls that check a rule still fires on the plain case, and one is a negative control on which the engine must stay silent, and does. Table 9 lists them; there is no mismatch against expectation.

Table 9

Evasion suite: seven evasion attempts (three caught, four blind spots), three positive controls and one negative control, with no mismatch against expectation.

Case	Rule	Kind	Outcome	What it shows
r1_ipv6_inner	R1	evasion	caught	Nested tunnel forwarding IPv6; the routable-inner guard accepts v6 as well as v4.
r1_outer_ext_header	R1	evasion	caught	Optional fields in the outer header shift the payload by four bytes; offset handling is correct.
r1_generic_gtp_nested	R1	evasion	caught	The nested header dissects as generic GTP rather than GTP-U; both are recognised.
r2_unseen_teid	R2	evasion	blind spot	A rogue sending on a never-seen identifier becomes its owner; nothing conflicts.
r2_spoofed_source_ip	R2	evasion	blind spot	Forging the base station's address keeps the (TEID, source) pair intact; limit of IP attribution.
r3_nonstandard_port	R3	evasion	blind spot	PFCP moved off port 8805 defeats a port-keyed rule.
r4_unlisted_core_nf	R4	evasion	blind spot	A core function outside the configured address set is invisible.
r3_pfcp_standard	R3	control	caught	PFCP on its standard port inside the tunnel.

Case	Rule	Kind	Outcome	What it shows
r3_sctp_inner	R3	control	caught	An inner packet carrying IP protocol 132.
r4_listed_core_nf	R4	control	caught	An inner packet aimed at a configured core function.
r1_noise_no_inner_ip	R1	negative control	not flagged	GTP-looking bytes with nothing routable behind them; silence is correct.

The four blind spots follow from rules that see only user-plane packets, not from accidents of implementation. r2_unseen_teid would close given a feed of allocated identifiers from the session management function, which an operator has and a passive lab tap does not; the TEID brute force in the 5G-CGAD captures (Section 7.1) and the two misses in live run L2 (Section 7.2) are instances of it. r2_spoofed_source_ip needs source authentication on N3, such as IPsec. The r3 case is the standing weakness of port-keyed detection and would need analysis of payload structure. The r4 case is a configuration problem rather than a detection one.

6.4. Comparison with Zeek and Suricata

We ran Zeek 8.2.2 with its default scripts and Suricata 8.0.7 with the Emerging Threats Open ruleset (53,289 rules loaded) over one capture per class, written from the same generator by eval/comparator_pcaps.py, and decoded the same captures with tshark 4.6.6. Table 10 gives the outcome. Zeek decapsulates GTPv1 and logs the inner connections of the benign, PFCP, SCTP and inner-to-core packets, so the evidence R3 and R4 read is present in its logs, but no default script raises a notice on any of them. For the IP-wrapped nested form it reports an analyzer violation named GTP-in-GTP. For the bare form it logs no inner connection and no second tunnel; the only trace is the generic violation “non-IP packet in GTPv1”, which says that the payload is not IP but not that it is a tunnel. For TEID re-sourcing it logs a second tunnel from the rogue address and the generic weird tunnel_max_changes_per_connection_exceeded. Suricata raised no alert on any class: it does not decode GTP-U, and the GTP rules in the ET Open set target GTP-C. tshark shows the bare-nested payload as T-PDU data and decodes every other class.

A Zeek policy script could implement R3 and R4 from the logged inner connections. R1 is different: for the bare form the information it needs never reaches the script layer, because the analyzer stops at the non-IP payload.

Table 10

The same per-class captures under the engine, the naive baseline and three open tools. Zeek and Suricata ran with default configuration; Suricata loaded 53,289 ET Open rules.

Class	Engine	Baseline	Zeek 8.2.2	Suricata 8.0.7	tshark 4.6.6
Benign	none	none	inner flows logged, no GTP notice	no alert	decoded
Bare nested	R1	none	“non-IP packet in GTPv1”; no inner flow	no alert	T-PDU data
IP-wrapped nested	R1, R3, R4	R1, R3, R4	“GTP-in-GTP” violation	no alert	decoded
PFCP smuggling	R3, R4	R3, R4	inner UDP/8805 logged, no notice	no alert	decoded
SCTP smuggling	R3, R4	R3, R4	inner protocol 132 logged, no notice	no alert	decoded
Inner-to-core	R4	R4	inner TCP to core logged, no notice	no alert	decoded
TEID re-sourcing	R2	R2	second tunnel logged; generic weird	no alert	decoded

Table 11

Per-category breakdown of the 600 categorised benign packets at seed 1337 (Table 4 gives the corpus totals). Only the unstructured and handover categories are at risk of firing a rule.

Category	Packets	Category	Packets	Category	Packets
web_tls	208	rtp_voip	35	handover	12
web_http	94	fragmented	34	unstructured	11
quic	78	ipv6	27	ntp	11
dns	57	icmp	23	ip_options	10

6.5. False positives and the handover ablation

Table 11 gives the benign composition at seed 1337, because a false-positive rate is only as informative as the traffic it was measured against. Most of it is ordinary web and DNS traffic that no byte-pattern rule is at risk of flagging. Only two categories could plausibly produce a false positive: unstructured payloads, which target R1, and handover packets, which target R2. At seed 1337 there are 23 such packets, 11 and 12, and none is flagged, which bounds the rate on them only below 0.143. Over the held-out seeds there are 334, again with none flagged, for an upper bound of 0.0114 (Table 7). The unit test that drives 200 unstructured payloads through R1, and the 6.2 million benign packets of Section 7.1, are stronger evidence.

With the base station allowlist configured, the false-positive rate is 0.0 and no benign packet in any of the twelve categories is flagged. Removing the allowlist raises it to 0.0083, six false positives out of 720 benign packets, and every one of the six is a handover. This is the expected failure and the reason the allowlist exists: an Xn or N2 handover legitimately moves a tunnel endpoint identifier from one base station to another, which is byte for byte what re-sourcing looks like. The allowlist suppresses transitions between known base stations while still flagging an identifier re-sourced from outside the pool. That residual rate is small but systematic. Without an accurate base station inventory R2 has no basis for moving ownership, so an operator should expect a handed-over session to keep raising alerts for as long as it sends from the new base station.

6.6. Processing cost

Per-packet cost across the full dissect and evaluate cycle, pooled over five passes of the seed-1337 corpus (eval/latency.py), is shown in Fig. 6: mean 716 µs (standard deviation of the five run means 44 µs), median 602 µs, 95th percentile 1,295 µs and 99th percentile 1,944 µs, or about 1,400 packets per second on one core of the development host. The maximum, 18.1 ms, is a single outlier. Dissection takes about 73% of the cost, so timing the rules alone would understate what a passive tap pays. Absolute values depend on the host; the classification results do not.

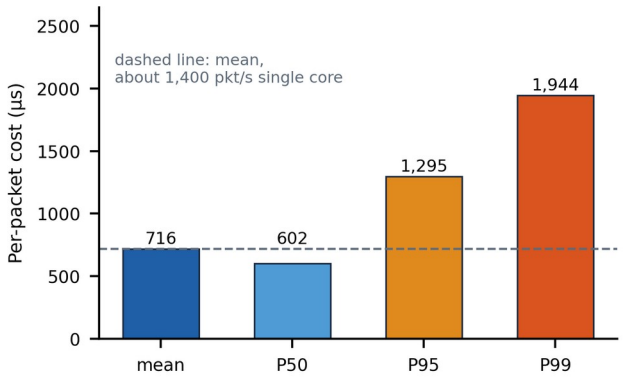


Fig. 6. Per-packet cost across the full dissect and evaluate cycle, pooled over five passes of the 1,320-packet corpus. Dissection is about 73% of the total, so timing rule evaluation alone understates what a passive tap pays.

7. Independent-data and live evaluation

7.1. The 5G-CGAD captures

The 5G-CGAD dataset [25] was produced by other researchers on a different testbed, with Open5GS on Kubernetes, 30 UERANSIM subscribers and three base stations, and we used its packet captures without modification. eval/external_eval.py streams each archive, keeps the frames that carry GTP-U, and scores the engine and the naive baseline together. The core addresses for R4, 10.42.0.64 and 10.42.0.208, were identified from the captures, and no base station allowlist was configured, so R2 ran in its strict mode. The dataset labels traffic per scenario and per flow rather than per packet, so an attack capture also carries that scenario's background traffic, and the fraction of its packets flagged is not a recall. Table 12 gives the results.

The benign captures are the main result. Across 6,218,456 GTP-U packets from 30 subscribers the engine raised no finding, which bounds its false-positive rate on this traffic below 6.2×10^{-7} (Wilson 95%). We did not generate this traffic, and its G-PDUs carry the extension header that real N3 traffic uses: the flag was set on all 345,054 G-PDUs in the first three capture files.

In the GTP encapsulation scenario every one of the 290,225 GTP-U packets is nested, and every one is flagged, by R1 and R3. The naive baseline flags them too, because the scenario uses the IP-wrapped form, which the dissector decodes. The dataset contains no bare-nested packet, so it tests the engine's false-positive behaviour and its handling of real framing, not the re-parse. In the malformed scenario R2 flags 176,903 of the 176,913 uplink packets: they reuse TEID 0x3039 from more than 32,000 forged source addresses, so every packet from a source other than the first is a re-sourcing. The downlink replies are not flagged. The engine has no rule for malformed headers as such.

The engine does not detect TEID brute force. In the two brute-force scenarios R2 fires on 6.4% and 1.2% of GTP-U packets, where the same identifier reaches the same receiver from more than one source address, but it has no rule for the enumeration itself, a stream of previously unseen identifiers; this is blind spot r2_unseen_teid (Table 9). The two denial-of-service scenarios are volumetric and outside the engine's scope. It flags 109 packets in the DDoS captures, all GTP-C carried inside the tunnel (R3), and one packet in the intra-UPF captures, a GTP-U datagram inside the tunnel (R3). The baseline raises R1 on three further intra-UPF packets on which the engine does not.

Table 12

Engine and naive baseline on the 5G-CGAD packet captures [25]. Labels are per scenario, so the flagged fraction of an attack capture is not a recall. R2 in strict mode; core addresses 10.42.0.64 and 10.42.0.208.

Scenario	GTP-U packets	Engine flagged	Rules	Baseline flagged	Reading
Benign	6,218,456	0 (upper 6.2e-7)	-	0	no false positive
GTP encapsulation (IP-wrapped)	290,225	290,225 (1.0)	R1, R3	290,225	detected; baseline also detects
Malformed GTP	353,136	176,903 (0.501)	R2	176,903	forged sources on one TEID
TEID brute force, G-PDU	878,333	56,002 (0.064)	R2	56,002	enumeration not detected
TEID brute force, several message types	10,054,554	123,931 (0.012)	R2	123,931	enumeration not detected
Intra-UPF DoS	426,949	1	R3	3	volumetric, out of scope
DDoS	11,561,011	109	R3	109	volumetric, out of scope

7.2. Live runs with three subscribers

The live path sends genuine traffic. Three UERANSIM subscribers register with the Open5GS core over NGAP and NAS, establish IPv4 PDU sessions and ping an external address through their tunnels for the whole run, while the attacker container writes the generator's packets onto N3 and the detector sniffs passively in the core's namespace. Two sequences were run on the cloud host with seeds not used before: L1, seed 2026, 400 malicious and 100 benign packets on TEID 0x2, and L2, seed 4242, 600 and 200 on TEID 0x7. Because the generator is seeded, writing the same sequence to a file instead of sending it reproduces the packets and their labels, and in both runs the ordered stream of live findings is identical, finding for finding, to an offline pass over that file (565 findings for L1, 853 for L2). The labels therefore score the live runs packet by packet. Table 13 sets them beside the development corpus.

L1 is classified without error. L2 has two false negatives and no false positive, for recall 0.9967. Both misses are the first two packets of the sequence, TEID re-sourcing packets that reached the detector before any packet from the legitimate base station on TEID 0x7. R2 binds an identifier to the first source it sees, so the rogue became the owner, and the base station then reclaimed the identifier without a finding, as the allowlist intends. This is the cold-start case of blind spot r2_unseen_teid; we report it rather than change the rule after seeing a held-out result. Over both runs the engine detects 998 of 1,000 malicious packets (95% interval 0.993 to 0.999), with no false positive on 300 benign ones (upper bound 0.0126). The real subscriber traffic, 12,136 packets in L1 and 19,390 in L2, raised no finding.

Development used one subscriber and one 500-packet sequence at seed 1337. That sequence exposed two flaws in the first R2 implementation, which handed an identifier to whichever source sent last and keyed ownership on the identifier alone. Under that implementation the run produced 608 findings, 146 of them from R2, and misclassified 31 packets: 16 benign packets that raised R2 and 15 spoofs that raised nothing. The two counts measure different things. Most of the 146 R2 findings were on malicious packets and correct; the 31 are the packets whose verdict was wrong. R2 was corrected as described in Section 5.4, and the corrected rule is the one evaluated here. Because the correction was made on that sequence, its later score is not counted as held-out evidence.

Table 13

Development corpus and the two held-out live runs. Each live finding stream is identical to an offline pass over the same sequence. Raw counts differ with corpus size and composition.

	Development, seed 1337 (offline)	L1 live, seed 2026	L2 live, seed 4242
R1 nested tunnel	120	79	131
R2 TEID re-sourcing	120	93	91
R3 control-plane smuggling	240	165	255
R4 inner-to-core	360	228	376
Total findings	840	565	853
Malicious / benign packets	600 / 720	400 / 100	600 / 200
Precision / recall	1.0 / 1.0	1.0 / 1.0	1.0 / 0.9967
False-positive rate	0.0	0.0	0.0
Real subscriber packets (findings)	-	12,136 (0)	19,390 (0)

Table 13 shows that the offline results survive real transmission and passive capture beside real sessions. It does not show independence from the rule design, since the live packets come from the same generator; that independence is what Section 7.1 provides, for the classes the 5G-CGAD captures contain.

7.3. Dissector source code and versions

The claim that default dissection does not decode the bare form rests on the source code of three tools, read at the versions used. In Scapy, GTP_U_Header.guess_payload_class is identical in releases 2.4.4 and 2.7.0: after any extension header, a G-PDU payload whose first byte is 0x45 to 0x4E is decoded as IPv4, one whose high nibble is 6 as IPv6, and anything else as PPP; no branch returns a GTP-U layer. In Wireshark 4.6.6 [33], a T-PDU is offered under the default preference to the heuristic dissectors registered on gtp.tpdu, an IP heuristic that requires version 4 or 6 and a consistent length and an Ethernet heuristic; no GTP heuristic is registered, so the nested header is shown as T-PDU data. In Zeek 8.2.2 [29], the GTPv1 analyzer forwards a G-PDU payload only if its version nibble is 4 or 6 and otherwise raises the violation “non-IP packet in GTPv1”; its GTP-in-GTP violation is reached only when a GTP packet arrives over UDP inside an existing GTPv1 tunnel, which is the IP-wrapped form. The three implementations were written independently and dispatch the same way on the first payload byte, which is why they share the blind spot. The source code of commercial firewalls is not available, so their handling of the bare form is unverified (Section 8).

The engine does not depend on that dispatch, because it recovers the payload bytes itself (Section 5.3). To check this, the development corpus at seed 1337 and the held-out corpus at seed 101 with session containers were scored under Scapy 2.4.4 on the development host and Scapy 2.7.0 on the cloud host. The ordered finding streams are identical under the two releases, 840 findings for each corpus.

8. Limitations and threats to validity

The development and held-out corpora are synthetic and their class balance is not realistic: roughly 55% of each is benign, far from the proportion on a production N3 interface. The 5G-CGAD benign captures address this in part for false positives, with 6.2 million packets from 30 subscribers and none flagged, but they come from another emulated testbed with Open5GS and UERANSIM rather than from an operator network, and they do not bound the rate under real handset diversity, roaming or slicing.

The most important scope limit concerns the bare form itself. It is reachable only from an on-path position on N3 or on an Unstructured or Ethernet PDU session, neither of which was exercised by a real device here; on an ordinary IPv4 or IPv6 session the base station encapsulates the subscriber's packet whole, so the first byte after the outer header is 0x4X or 0x6X. The IP-wrapped form an ordinary session can send is decoded by default dissection (Table 8) and reported by Zeek (Table 10). No public dataset we know of contains the bare form; the 5G-CGAD nested scenario is IP-wrapped. The engine's detection of the bare form has therefore been measured only on packets we generated, in held-out seeds and live runs, and the claim is limited to that.

The dissector claim covers the three open implementations whose source we read. Juniper documents a GTP-in-GTP check and Fortinet states that its firewall cannot open a nested tunnel, but neither implementation can be inspected and we had access to neither. The released per-class captures (eval/comparator_pcaps.py) let anyone with such a device repeat the check.

The R1 gate is a heuristic, hardened but not cryptographic. The three checks defeat the benign collisions and the crafted evasions reported here, but an adversary who understands all three could in principle build a nested tunnel that satisfies every one of them while still being shaped to confuse a downstream parser differently from ours. That residual surface is untested and is inherent to any byte-pattern heuristic.

The re-parse is also single-level. It recovers one nested header that forwards a routable inner packet, which covers the corpus construction, but an adversary who nests twice defeats it, and so does the zero-length nested G-PDU of [2] in the bare form, which the length gate rejects by design. Extending to bounded recursion is straightforward, and the bound would have to be explicit, since unbounded re-parsing driven by attacker-controlled length fields would turn the detector into the denial of service it exists to detect. It is not implemented, because there is no evidence that deeper nesting occurs.

Each rule carries limits beyond the evasions in Table 9. R2 binds an identifier to the first source it sees, scoped to the receiving endpoint, with no ageing or expiry. A rogue that transmits before the legitimate base station becomes the owner, which is the cause of the two misses in live run L2; an identifier recycled after a session ends is an unmeasured source of false positives; and R2 has no notion of TEID enumeration, so the brute force in 5G-CGAD is not detected. R3 keys on ports and protocols. R4 depends on correct manual configuration, and a wrong entry silently disables it. IPv6 inner coverage is partial (Section 4).

The live runs used three subscribers on one core and one base station, the test public land mobile network, no roaming or slicing, and two sequences. Handover is modelled synthetically with two base station addresses rather than run on a core with several stations. Timing was measured on one core of one lab machine with no competing load, so the figure of about 1,400 packets per second is not validated against production rates, and no multi-core or sustained-load test was done.

Finally, the engine and the generator share authors, so the risk that the generated corpus tests what the engine happens to detect is real. Held-out seeds and fresh live sequences reduce the risk of tuning to particular packets, but not that of a shared understanding of the classes. The ablation holds the corpus fixed and varies only the parser, the evasion suite was written to defeat the engine, and the 5G-CGAD captures were produced by others; for the bare form, the last of these is not yet available.

9. Conclusion and future work

GTP-U tunnels carry subscriber traffic with no protection of their own, and published disclosures show that they can be abused. For one abusive construction, a GTP-U header placed directly after the outer one, the obstacle to detection is the packet dissector. Scapy, Wireshark and Zeek choose a G-PDU's inner protocol from the version nibble of its first byte and have no path that decodes a nested GTP-U header, so a detector built on them misses every such packet and reports no error. Removing the targeted re-parse from our engine, with everything else fixed, drops recall on that class from 1.0 to 0.0. The IP-wrapped form, which an ordinary session can send, is decoded by default, caught by the naive baseline and reported by Zeek.

The engine re-dissects the payload after the outer header and its extension headers, behind a byte-level gate. On held-out seeds with realistic N3 framing it makes no error over 13,203 packets; without the extension-header handling it misses every bare-nested packet, which the development corpus could not show. On the independent 5G-CGAD captures it raises no finding on 6.2 million benign packets and flags every nested packet, and it does not detect TEID brute force. Two fresh live runs with three subscribers reproduce the offline finding streams exactly, with two misses that follow from R2's first-source binding. Zeek reports only the IP-wrapped form, and Suricata with the ET Open rules reports neither.

Four directions follow. The bare form should be sought in operator traffic, or produced on a testbed that supports Unstructured or Ethernet PDU sessions, which would test the main claim on data we did not generate. A feed of allocated tunnel endpoint identifiers from the session management function would close the r2_unseen_teid blind spot and the cold-start miss, and would support a rule for TEID enumeration. A learned or flow-based detector run on the same corpora would show whether flow features separate nested packets. And the same reading of dissector dispatch could be applied to other tunnelling protocols that allow nesting.

Ethics and responsible use

Every result that involves generated attack traffic was produced inside a self-contained laboratory on a private Docker network, on the development host or on a cloud host under the authors' control, under the test public land mobile network 999/70, which is reserved for testing and belongs to no operator. The only traffic that left the laboratory was benign ICMP echo from the emulated subscribers to a public address, translated to the host's address on its way out; no generated abuse traffic left it. The 5G-CGAD captures were used under their CC BY 4.0 licence and contain only emulated subscribers. Generating GTP-U abuse traffic against a live mobile operator's infrastructure is unlawful in most jurisdictions; it was not done, attempted or intended in this work, and the released generator must not be pointed at any network its user does not own or is not explicitly authorised to test.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The testbed, corpus generator, detection engine, evaluation scripts, evasion suite and the result files of every run reported here are available under the MIT licence at https://github.com/kobbycyber/gtp-abuse-detector. The 5G-CGAD captures are available from the University of Oulu, doi:10.23729/fd-b9459ea4-88a7-35c2-992e-f178a294c12d [25].

References

[1] 3GPP, General Packet Radio System (GPRS) Tunnelling Protocol User Plane (GTPv1-U), TS 29.281, 3rd Generation Partnership Project, Release 18, 2024.

[2] S. I. Salim, Attacks on 5G infrastructure from users' devices: GTP-U tunnel abuse and CVE-2021-45462, Trend Micro Research, 2023.

[3] S. I. Salim, Attacks on 5G infrastructure from user devices: ASN.1 vulnerabilities in 5G cores, Trend Micro Research, 2023.

[4] S. I. Salim, Outside looking in: how a packet reflection vulnerability could allow attackers to infiltrate internal 5G networks, Trend Micro Research with CTOne and the Telecom Technology Center, 2023.

[5] P. Biondi and the Scapy community, Scapy: packet crafting and manipulation library, https://scapy.net, accessed 2026.

[6] 3GPP, Security architecture and procedures for 5G System, TS 33.501, 3rd Generation Partnership Project, Release 18, 2024.

[7] GSMA, FS.37 GTP-U Security, GSM Association, 2022.

[8] M. Harvanek, J. Bolcek, J. Kufa, L. Polak, M. Simka, R. Marsalek, Survey on 5G physical layer security threats and countermeasures, Sensors 24 (17) (2024) 5523.

[9] P. Scalise, M. Boeding, M. Hempel, H. Sharif, J. Delloiacovo, J. Reed, A systematic survey on 5G and 6G security considerations, challenges, trends, and research areas, Future Internet 16 (3) (2024) 67.

[10] M. Bannat Wala, M. Kiran, 5G network security practices: an overview and survey, arXiv:2401.14350 (2024).

[11] S. T. Aiello, B. P. Rimal, F. T. Sheldon, Y. Wang, Investigating security vulnerabilities in 5G control and user planes: attack patterns and protection strategies, Journal of Cybersecurity and Privacy 6 (1) (2026) 37. doi:10.3390/jcp6010037.

[12] R. Patil, Z. Tian, M. Gurusamy, J. McCloud, 5G core network control plane: network security challenges and solution requirements, Computer Communications 229 (2024) 107982. doi:10.1016/j.comcom.2024.107982.

[13] Z. Salazar, H. N. Nguyen, W. Mallouli, A. R. Cavalli, E. Montes de Oca, 5Greplay: a 5G network traffic fuzzer, application to attack injection, in: Proceedings of the 16th International Conference on Availability, Reliability and Security (ARES 2021), ACM, 2021, article 106, pp. 1-12. doi:10.1145/3465481.3470079.

[14] S. Lee, Open5GS: open source implementation of the 5G core and EPC, https://open5gs.org, accessed 2026.

[15] A. Gungor, UERANSIM: open source 5G UE and RAN simulator, https://github.com/aligungr/UERANSIM, accessed 2026.

[16] M. Barbosa, M. Silva, E. Cavalcanti, K. Dias, Open-source 5G core platforms: a low-cost solution and performance evaluation, in: 2025 International Conference on Information Networking (ICOIN), IEEE, 2025, pp. 99-104. doi:10.1109/ICOIN63865.2025.10992769.

[17] D. Pineda, R. Harrilal-Parchment, K. Akkaya, A. Ibrahim, A. Perez-Pons, Design and analysis of an open-source SDN-based 5G standalone testbed, in: IEEE INFOCOM 2023 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2023, pp. 1-6.

[18] M. Rouili, N. Saha, M. Golkarifard, M. Zangooei, R. Boutaba, E. Onur, A. Saleh, Evaluating open-source 5G SA testbeds: unveiling performance disparities in RAN scenarios, in: NOMS 2024 IEEE Network Operations and Management Symposium, IEEE, 2024, pp. 1-6.

[19] T. Horstmann, D. Brunke, T. Kremeyer, M. Wilmes, G. Schneider, J. Sturm, H. Koenig, M. Rademacher, open5Gcube: a modular and usable framework for mobile network laboratories, in: Proceedings of the 29th ITG-Symposium on Mobile Communication, 2025. arXiv:2505.14501.

[20] G. Amponis, P. Radoglou-Grammatikis, T. Lagkas, S. Ouzounidis, M. Zevgara, I. Moscholios, S. Goudos, P. Sarigiannidis, Generating full-stack 5G security datasets: IP-layer and core network persistent PDU session attacks, AEU International Journal of Electronics and Communications 171 (2023) 154913.

[21] G. Amponis, P. Radoglou-Grammatikis, T. Lagkas, W. Mallouli, A. Cavalli, D. Klonidis, E. Markakis, P. Sarigiannidis, Threatening the 5G core via PFCP DoS attacks: the case of blocking UAV communications, EURASIP Journal on Wireless Communications and Networking 2022 (1) (2022) 124.

[22] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, M. Ylianttila, 5G-NIDD: a comprehensive network intrusion detection dataset generated over 5G wireless network, arXiv:2212.01298 (2022).

[23] N. Lassoued, I. Filali, R. Ejbali, Supervised machine learning-based intrusion detection for 5G networks: evaluation on the 5G-NIDD dataset, Computers 15 (6) (2026) 362.

[24] Y.-E. Kim, Y.-S. Kim, H. Kim, Effective feature selection methods to detect IoT DDoS attack in 5G core network, Sensors 22 (10) (2022) 3819. doi:10.3390/s22103819.

[25] S. P. Wengappuli Arachchige, Y. Siriwardhana, T. Hewa, 5G Core GTP-U Attack Dataset (5G-CGAD), University of Oulu, 2025, CC BY 4.0. doi:10.23729/fd-b9459ea4-88a7-35c2-992e-f178a294c12d.

[26] Juniper Networks, Understanding GTP-U inspection, Junos OS for SRX Series documentation, https://juniper.net/documentation/en_US/junos12.1x47/topics/concept/security-gtp-u-inspection-understanding.html, accessed 2026.

[27] Fortinet, Configuring encapsulated filtering in FortiOS Carrier, FortiOS 6.0 Handbook, https://docs.fortinet.com/document/fortigate/6.0.0/handbook/418319, accessed 2026.

[28] Palo Alto Networks, Objects > Security Profiles > Mobile Network Protection, PAN-OS web interface help, https://docs.paloaltonetworks.com/ngfw/help/11-1/objects/objects-security-profiles-mobile-network-protection, accessed 2026.

[29] The Zeek Project, Zeek network security monitor, version 8.2.2, GTPv1 packet analyzer (src/packet_analysis/protocol/gtpv1), https://zeek.org, accessed 2026.

[30] Open Information Security Foundation, Suricata, version 8.0.7, https://suricata.io, with the Emerging Threats Open ruleset, https://rules.emergingthreats.net, accessed 2026.

[31] Cisco, Snort 3 inspector reference: gtp_inspect, https://docs.snort.org, accessed 2026.

[32] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209-212.

[33] Wireshark Foundation, Wireshark 4.6.6, GTP and IP dissectors (epan/dissectors/packet-gtp.c, packet-ip.c), https://www.wireshark.org, accessed 2026.